

UnoLib documentation

spi.pas

version 06/09/2026

The spi.pas module provides implementation of the TSPI object and TSPISettings object, allowing communication using the Serial Peripheral Interface (SPI) hardware available in AVR microcontrollers. It allows configuration of clock frequency, bit order and SPI mode, as well as safe access to the SPI bus when interrupts are used.

Since FPC for AVR does not allow dynamic object creation, the global object `_SPI` declared by the module is ready to use and does not require the use of a constructor.

TSPISettings

The TSPISettings object stores SPI communication settings used by BeginTransaction.

TSPISettings methods
<pre>constructor Init (clock: UInt32; bitOrder, dataMode: UInt8);</pre>
Creates a settings object using the specified clock frequency, bit order and SPI mode.
<i>Parameters</i> <i>clock</i> – desired SPI clock frequency in Hz <i>bitOrder</i> – MSBFIRST or LSBFIRST <i>dataMode</i> – SPI_MODE0, SPI_MODE1, SPI_MODE2 or SPI_MODE3
<pre>constructor Init;</pre>
Creates a settings object using default parameters: clock = 4000000 Hz bitOrder = MSBFIRST dataMode = SPI_MODE0

TSPI

TSPI methods
<pre>procedure _begin;</pre>
Initializes the SPI hardware and configures the controller as SPI master. The SS pin is configured as output and driven high. The MOSI and SCK pins are configured

as outputs. Calling `_begin` multiple times is allowed.

Note: In Arduino sources the method is called *begin*.

```
procedure _end;
```

Disables the SPI hardware.

Note: In Arduino sources the method is called *end*.

```
procedure UsingInterrupt (interruptNumber: UInt8);
```

Registers an interrupt that may use SPI communication. This allows `BeginTransaction` to temporarily disable the specified interrupt and avoid SPI bus conflicts.

Parameters

interruptNumber – interrupt number (for example 0 for INT0 or 1 for INT1)

```
procedure NotUsingInterrupt (interruptNumber: UInt8);
```

Removes a previously registered interrupt from SPI transaction management.

```
procedure BeginTransaction (settings: TSPISettings);
```

Starts an SPI transaction and applies communication settings stored in the settings object. When interrupt management is enabled using `UsingInterrupt`, the corresponding interrupts are temporarily disabled until `EndTransaction` is called.

Parameters

settings – SPI communication settings

```
procedure EndTransaction;
```

Ends the current SPI transaction and restores interrupt state.

```
function Transfer (data: UInt8): UInt8;
```

Transfers one byte through the SPI bus.

The specified byte is transmitted through MOSI while a byte is simultaneously received through MISO.

Return value

Byte received from the SPI slave device.

```
function Transfer16 (data: UInt16): UInt16;
```

Transfers a 16-bit value through the SPI bus. The transfer order depends on the currently selected bit order setting.

Return value

16-bit value received from the SPI slave device.

```
procedure Transfer(buf: Pointer; count: SizeUInt);
```

Transfers multiple bytes through the SPI bus. The transmitted data is replaced by data received from the SPI slave device.

Parameters

buf – pointer to the data buffer

count – number of bytes to transfer

```
procedure SetBitOrder(bitOrder: UInt8);
```

Sets SPI bit order.

Note: This method is provided for compatibility. New applications should use *BeginTransaction* and *TSPISettings*.

```
procedure SetDataMode(dataMode: UInt8);
```

Sets SPI mode. Possible values: SPI_MODE0, SPI_MODE1, SPI_MODE2, SPI_MODE3

Note: This method is provided for compatibility. New applications should use *BeginTransaction* and *TSPISettings*.

```
procedure SetClockDivider(clockDiv: UInt8);
```

Sets SPI clock divider. Possible values: SPI_CLOCK_DIV2, SPI_CLOCK_DIV4, SPI_CLOCK_DIV8, SPI_CLOCK_DIV16, SPI_CLOCK_DIV32, SPI_CLOCK_DIV64, SPI_CLOCK_DIV128

Note: This method is provided for compatibility. New applications should use *BeginTransaction* and *TSPISettings*.

Example 1

```
var
    Settings: TSPISettings;
    Value: UInt8;
begin
    _SPI._begin;

    Settings.Init(1000000, MSBFIRST, SPI_MODE0);

    _SPI.BeginTransaction(Settings);
    Value:= _SPI.Transfer($55);
```

```
_SPI.EndTransaction;  
end;
```

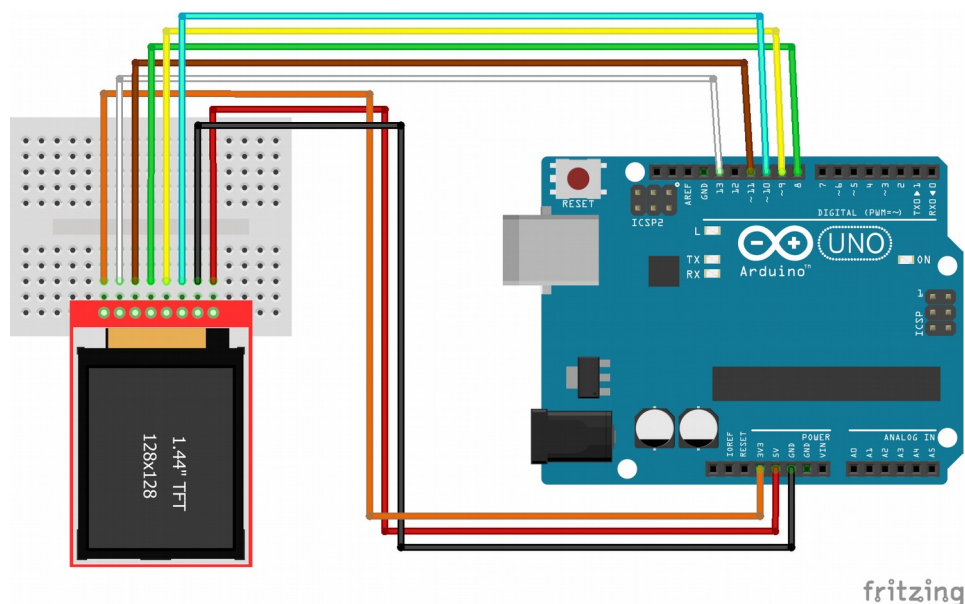
Example 2

The following example demonstrates the use of the SPI interface with an ILI9163-based 128×128 TFT display connected to an Arduino Uno-compatible board. The example performs the following operations:

- Initializes the SPI hardware.
- Configures SPI communication using: MSBFIRST bit order, SPI_MODE0 and SPI_CLOCK_DIV2 clock divider
- Resets and initializes the display controller.
- Sends display commands and data through the SPI bus.
- Fills the screen with several colors.
- Transfers a 128×128 indexed bitmap stored in program memory.

The example also demonstrates a common SPI usage pattern where the Chip Select (CS) signal remains active during a long sequence of data transfers. This allows efficient transmission of display data without repeatedly enabling and disabling the SPI device.

The example also demonstrates techniques commonly used on resource-constrained microcontrollers. In particular, it shows how indexed bitmaps and program memory (Flash) can be used to display relatively large images while minimizing SRAM and Flash memory usage. The bitmap is stored as an indexed image (8 bits per pixel) rather than a full 16-bit RGB565 image. A 128×128 RGB565 bitmap would require 32 KB of storage, while the indexed representation requires only 16 KB plus a 512-byte color palette. Bitmap and palette data are accessed from Flash memory at runtime using *pgm_read_byte* and *pgm_read_word* functions.



fritzing

```

program TestILI9163;

{
  ILI9163 TTF SPI 128x128 Display

  TTF Arduino Uno
  -----

  LED      -> 3.3V
  SCK      -> D13
  SDA      -> D11
  A0       -> D8
  RESET    -> D9
  CS       -> D10
  GND      -> GND
  VCC      -> 3.3V

  written 2026 by ackarwow
}

{$IFDEF AVRPascal}
  {$IF NOT (DEFINED(atmega328p) or DEFINED(arduinouno) or
  DEFINED(arduinonano))}
    {$Fatal Invalid controller type, expected: atmega328p, arduinouno, or
  arduinonano}
  {$ENDIF}
{$ELSE}
  {$IF NOT (DEFINED(fpc_mcu_atmega328p) or DEFINED(fpc_mcu_arduinouno) or
  DEFINED(fpc_mcu_arduinonano))}
    {$Fatal Invalid controller type, expected: atmega328p, arduinouno, or
  arduinonano}
  {$ENDIF}
{$ENDIF}

//{$mode objfpc} Result, overloading etc. not needed

uses
  defs,
  timer, digital,
  spi;

{$i UnoLibPalette.inc} //BGR565 format
{$i UnoLibBitmap.inc}

const
  TFT_CS    = 10;
  TFT_DC    = 8;
  TFT_RESET = 9;

//after
https://github.com/ccrause/fpc-avr/blob/33856a5abc210138f61e9c13f567a8f4ac7e1486/src/library/progmem.pas

function pgm_read_byte(const address: pointer): byte; assembler;
nostackframe;
asm
  movw ZL, r24
  lpm r24, Z
end['r24', 'r25', 'r30', 'r31'];

```

```

function pgm_read_word(const address: pointer): UInt16; assembler;
nostackframe;
asm
    movw ZL, r24
    lpm r24, Z+
    lpm r25, Z
end['r24', 'r25', 'r30', 'r31'];

procedure TFT_WriteCommand(Cmd: Byte);
begin
    DigitalWrite(TFT_DC, LOW);
    DigitalWrite(TFT_CS, LOW);
    _SPI.Transfer(Cmd);
    DigitalWrite(TFT_CS, HIGH);
end;

procedure TFT_WriteData(Data: Byte);
begin
    DigitalWrite(TFT_DC, HIGH);
    DigitalWrite(TFT_CS, LOW);
    _SPI.Transfer(Data);
    DigitalWrite(TFT_CS, HIGH);
end;

procedure TFT_FillScreen(Color: UInt16);
var
    i: UInt16;
begin
    DigitalWrite(TFT_CS, LOW);

    DigitalWrite(TFT_DC, LOW);

    // RAM write
    // After sending command $2C the controller expects
    // a continuous stream of pixel data.
    // CS must remain LOW during the whole transfer.
    _SPI.Transfer($2C); // RAMWR

    DigitalWrite(TFT_DC, HIGH);

    for i:=0 to 16383 do
        begin
            _SPI.Transfer(Color shr 8);
            _SPI.Transfer(Color and $FF);
        end;

    DigitalWrite(TFT_CS, HIGH);
end;

procedure TFT_DrawBitmap128;
var
    i: UInt16;
    idx: Byte;
    Color: UInt16;
begin
    // Draw 128x128 indexed bitmap stored in program memory.
    // Bitmap - 16384 bytes (1 byte per pixel)
    // Palette - 256 entries (BGR565)

```

```

// Both tables are stored in .progmem and accessed
// through pgm_read_byte / pgm_read_word.

DigitalWrite(TFT_CS, LOW);

DigitalWrite(TFT_DC, LOW);

// RAM write
// After sending command $2C the controller expects
// a continuous stream of pixel data.
// CS must remain LOW during the whole transfer.
_SPI.Transfer($2C); // RAMWR

DigitalWrite(TFT_DC, HIGH);

for i:=0 to 16383 do
begin
    idx:=pgm_read_byte(@UnoLibBitmap[i]);
    Color:=pgm_read_word(@UnoLibPalette[idx]);

    _SPI.Transfer(Color shr 8);
    _SPI.Transfer(Color and $FF);
end;

DigitalWrite(TFT_CS, HIGH);
end;

begin
_SPI._begin;
_SPI.setBitOrder(MSBFIRST);
_SPI.setDataMode(SPI_MODE0);
_SPI.setClockDivider(SPI_CLOCK_DIV2);

PinMode(TFT_CS, OUTPUT);
PinMode(TFT_DC, OUTPUT);
PinMode(TFT_RESET, OUTPUT);

DigitalWrite(TFT_CS, HIGH);
DigitalWrite(TFT_DC, HIGH);
DigitalWrite(TFT_RESET, HIGH);

// Hardware reset
DigitalWrite(TFT_RESET, LOW);
Delay(20);

DigitalWrite(TFT_RESET, HIGH);
Delay(120);

// Software reset
TFT_WriteCommand($01);
Delay(150);

// Exit sleep mode
TFT_WriteCommand($11);
Delay(150);

// Pixel format = BGR 565 (16-bit)
TFT_WriteCommand($3A);
TFT_WriteData($05);

```

```

Delay(10);

// Display ON
TFT_WriteCommand($29);
Delay(150);

// Column address range: 0..127
TFT_WriteCommand($2A); // CASET
TFT_WriteData(0);
TFT_WriteData(0);
TFT_WriteData(0);
TFT_WriteData(127);

// Row address range: 0..127
TFT_WriteCommand($2B); // PASET
TFT_WriteData(0);
TFT_WriteData(0);
TFT_WriteData(0);
TFT_WriteData(127);

//BGR565
TFT_FillScreen($F800); //blue
Delay(500);

TFT_FillScreen($07E0); //green
Delay(500);

TFT_FillScreen($001F); //red
Delay(500);

TFT_DrawBitmap128;

while true do ;
end.

```